

RoboJuggler: Learning Dexterous Juggling through Reinforcement Learning

Rhea Rai, Abhinav Peri

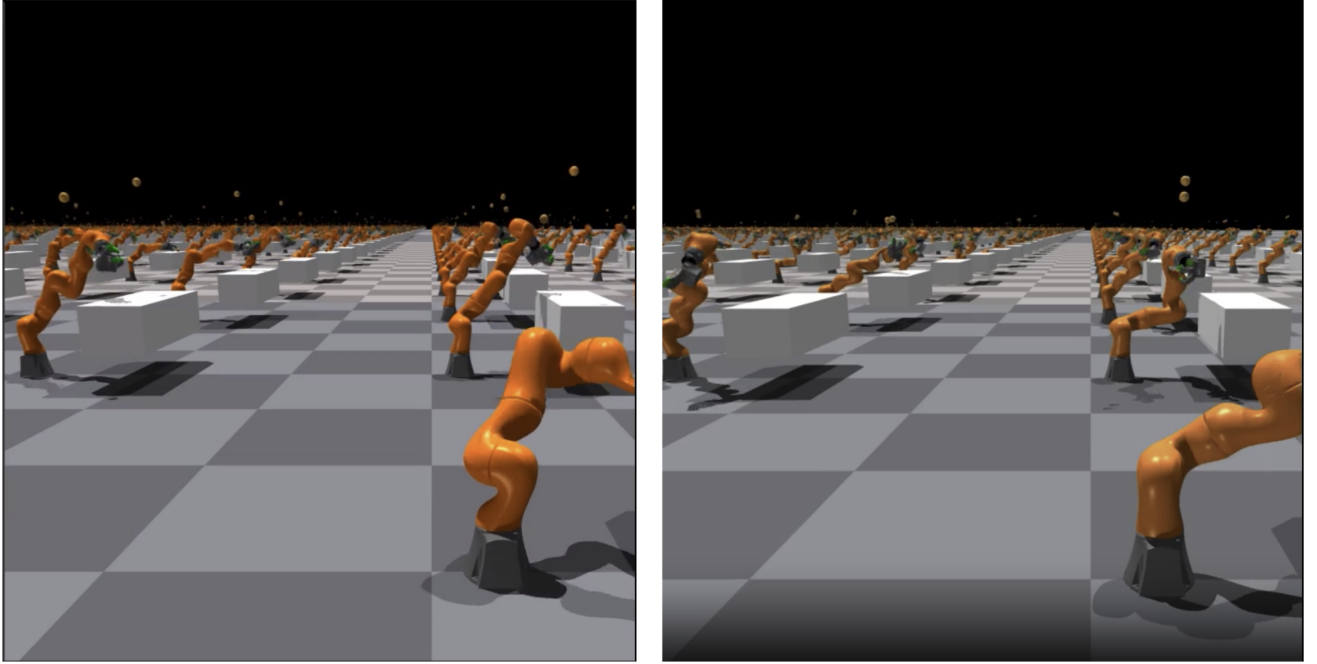


Fig. 1: An overview of the robotic juggling system used in this work. The system coordinates the 7-DoF KUKA arm with the Allegro hand to manipulate a spherical object via learned throwing and catching policies.

Abstract—Humans regularly learn and perform complex and nuanced motor skills, such as walking, playing sports, or manipulating objects with dexterity, that remain challenging benchmarks for even state-of-the-art robotic systems. Among these tasks, juggling represents a particularly compelling benchmark because it requires precise timing, coordinated manipulation, and dynamic control. Although robotic juggling has been demonstrated previously, most methods exist within simplified 2D simulators, using pre-programmed skills or expert demonstrations, use non-dexterous end-effectors, or rely on binary-reward or open-loop policies that limit adaptability and realism.

In this project, we leverage GPU-parallelized reinforcement learning in Isaac Gym to teach a robot hand-arm system with a human-like Allegro gripper to juggle objects—a task not commonly addressed in existing literature. We attempted juggling with up to three balls and achieved solid two-ball juggling performance, demonstrating sustained catches and throws. To support this, we designed a custom simulation environment with carefully crafted reward and state frameworks to scale and stabilize PPO, using domain-specific insights to iterate on control behavior, curriculum design, and reward sparsity. The result is a high-performing policy capable of executing multi-ball juggling in a physically realistic environment. This work demonstrates the feasibility of complex dexterous behaviors using standard RL algorithms, provided that environment engineering and parallel simulation are properly leveraged.

I. INTRODUCTION

Dexterous object manipulation remains one of the most formidable challenges in robotics. While humans can effortlessly perform simple skills, like lifting, throwing, catching, and manipulating object, or even more intricate tasks such as tying knots or assembling objects, they remain notoriously difficult for robotic systems to master. Among such tasks, juggling in particular is a uniquely demanding benchmark; it requires real-time coordination of perception, motor control, and object dynamics, all under dynamic conditions and with little room for error. A robot must not only grasp and throw objects but also anticipate their trajectories, time its actions precisely, and adapt quickly to variation. However, few existing works seriously attempt robotic juggling, and those that do often make heavy simplifications: using 2D simulators, low-DOF arms, binary reward signals, expert demonstrations, or open-loop behaviors that remove the learning aspect entirely.

In this project, we aim to push the boundary of what is possible using pure reinforcement learning in a high-fidelity simulator. We use NVIDIA Isaac Gym to simulate a robot composed of a 7-DOF KUKA arm and a 16-DOF Allegro hand, a dexterous gripper commonly used in manipulation research. Our environment builds on the Allegro-KUKA task

suite, but introduces new state features, dense reward shaping terms, and custom state information to enable learning juggling from scratch. We initially attempted to use robosuite, but the lack of GPU parallelization made it infeasible to conduct the scale of experimentation we needed. With Isaac Gym, we were able to parallelize hundreds of simulations and rapidly iterate over reward functions, curriculum steps, and object configurations.

Juggling can be formulated as a MDP problem, allowing it to be learned via reinforcement learning. With this insight, we trained our policies using Proximal Policy Optimization (PPO) as an on-policy RL algorithm. Over time, our policies learned meaningful juggling trajectories. With a single ball, we achieved consistent lift-toss-catch cycles. With two balls, we compared two variants: parallel throws (both tossed at once) and sequential throws (each thrown only after the previous is airborne), with the sequential variant proving more stable. A key technical innovation was our reward design, which combined lifting bonuses, throw detection, catching rewards, and hand-motion penalties to guide behavior across different juggling phases. We also introduced a ball scheduler mechanism to alternate focus between balls and gate reward timing, encouraging rhythmic behavior. While we attempted three-ball juggling, the policy failed to stabilize, highlighting the exponential complexity added with each object and the limitations of our current policy to generalize well.

Our final policy demonstrates reliable two-ball juggling in sequential motion with smooth control, repeated successful tosses, and realistic hand-arm coordination. It learns this entirely from scratch, without scripted priors or demonstrations. These results further demonstrate that deep reinforcement learning can produce robust dexterous behaviors for open-ended manipulation tasks like juggling. They also signify the importance of simulator fidelity, reward function engineering, and system-level design choices for training complex robotic skills.

II. RELATED WORK

A. Existing Work in Continuous Control with RL

Reinforcement learning (RL) has demonstrated significant success in continuous control tasks. *Proximal Policy Optimization (PPO)* [9] builds upon prior actor-critic algorithms, simplifying the optimization objective and enabling stable training. PPO has become a staple in modern reinforcement learning, demonstrating robust performance across MuJoCo control benchmarks [11]. We consider this representative of our task, since MuJoCo, like juggling, is a high precision, fine motor skill task. Off-policy methods like Deep Deterministic Policy Gradient (DDPG) [3] and Soft Actor-Critic (SAC) [2] have further enhanced sample efficiency compared to PPO, leveraging experience replay to reuse data effectively. Both DDPG and SAC have shown superior sample efficiency on MuJoCo and related robotics tasks, motivating their use as a foundation for our juggling task, alongside PPO as a baseline comparison.

B. Existing Work in Juggling Tasks

In "A physics-based juggling simulation using reinforcement learning" [1], the authors train a policy to juggle balls in a 2D physics simulation. By modeling the physics of the throwing and catching, the juggling routines are set up as planned throws and catches for each hand. Reinforcement Learning is then used to train the policy for throwing and catching, through the DDPG algorithm. For our purposes, we are most interested in their choice of reward function. For catching, they assign -1 reward for failure and a distance error between the target and actual catch. For throwing, the difference between actual throw point/throw velocity and target throw point/throw velocity is used as signal. We hypothesize that this is not fully generalizable to our 3D physics simulation, and propose using denser rewards, motivating our study of reward shaping methods to solve this task.

In "Learning Robotic Contact Juggling" [10], juggling is learned through a model based reinforcement learning method. They learn a model to predict the next state from the given state and action. From this, they run MPC to find a trajectory that minimizes tracking error from an expert demonstration. This paper demonstrates how model based methods combined with planning and expert demonstrations can be used to learn fine motor tasks. Although we are not going down this route, we find their results impressive, and keep note for broader context.

In "High Acceleration Reinforcement Learning for Real-World Juggling with Binary Rewards" [8], 2-ball juggling is trained with an open-loop, model-free policy with eREPS optimization on binary rewards. The paper is unique in its reward shaping, which assigns 1 for both ball heights above a given threshold, and 0 if not. Unlike traditional dense reward shaping, their binary reward function maximizes task success instead of optimizing for intermediate states to learn juggling. However, their end-effector used in this task is cup-like. We propose that to expand on this work and solve the juggling task with dexterous manipulation, we will need to incorporate more complex, denser reward signals.

C. Existing Work in Dexterous Hand Manipulation:

The methodology in "Learning Dexterous In-Hand Manipulation" [5] focuses on training a five-fingered robotic hand to reorient objects in-hand using PPO, without relying on any human demonstrations or imitation learning. This is trained fully in simulation and uses domain randomization to allow for a highly generalizable sim-to-real transfer. The methodology includes training a policy with Proximal Policy Optimization (PPO), with rewards assigned based on rotation angles of the object orientations at each time step, with additional rewards for goal state and penalties for dropping the object. This is relevant as we develop the reward shaping for our project, since we train a similarly dexterous end-effector to manipulate a ball object as it juggles. The work in "Solving Rubik's Cube with a Robot Hand" [6] builds further on the on the previous paper, and continues to show the use-case for purely RL approaches to dexterous manipulation, this time on Rubik's

cube manipulation. This establishes relevance and legitimacy in our hypothesis that we can approach robot juggling with a purely RL methodology, rather than relying on any demonstrations. The paper introduces the idea of automatic domain randomization (ADR), a methodology which progressively expands the range of environmental variations during training, allowing for better generalizability. In the future, as we seek to potentially generalize our project to transfer sim2real, these methodologies may also provide future context on how to best train a continuous task like juggling using RL to accurately perform in a real environment.

DexPBT [7] is an algorithm implemented on top of the IsaacGym simulation environment [4]. This method trains an LSTM model using PPO in the inner loop and PBT for the outer loop hyperparameter optimization. The PBT algorithm mutates hyperparameters dynamically across a population of agents to find the best schedule. This method enables much better exploration than vanilla RL methods, allowing faster learning of dexterous behaviors. This method was impossible to implement for the context of our project, as we lacked adequate GPU compute. However, given the complexities of learning the juggling task, we leave it as an avenue of future study for possibly faster convergence to stable juggling policies with adequate exploration.

D. Existing Work in Simulators

To train and test our policy, we experimented with two main simulation environments; Robosuite [12] and IsaacGym [4]. Robosuite [12] is a Mujoco [11] based physics simulator which we used as proof of concept as we begin to train smaller and intermediate tasks. However, most of our work was done using IsaacGym, a GPU-based physics simulator which has strong parallelism by leveraging vectorization in training.

III. DATA

For our project, we used a custom simulation environment built in NVIDIA’s Isaac Gym, specifically extending the AllegroKuka task framework to allow support of multi-ball juggling. The environment includes a 7-DoF KUKA arm mounted with an Allegro robotic hand, and multiple simulated rigid-body spheres (we will refer to as balls) designed to be juggled.

The physics simulation relies on Isaac Gym’s GPU-accelerated rigid-body dynamics. We initially considered using RoboSuite for this project, but ultimately chose Isaac Gym due to its GPU-based parallelization, which was important for scaling up training across many reward configurations and environment variants. The simulation was parameterized using YAML configuration files, which controlled reward scales, object spawning, episode lengths, and penalties. The juggling environment was initialized with 1 - 3 balls, though our most successful runs, which we will discuss in later sections, were done with 2-ball juggling.

We did not use any external datasets; instead, all data came from simulation. However, engineering our data for collection was necessary in two main ways. We engineered a custom

observation space that included fingertip-object distances and throw state flags. Our reward shaping also involved designing multiple reward signals (eg. lifting bonuses, verticality rewards, hand velocity penalties) to stabilize training.

This highly structured simulated data enabled us to train a robust juggling policy end-to-end using reinforcement learning.

IV. METHODS

Our approach was to train a policy using Reinforcement Learning to learn to Juggle. Although we tried various different methods, the most successful route was through aggressive reward shaping. Therefore, we spend a significant portion of the section discussing our findings in this realm.

Reward shaping can be very difficult to get right. We iterated over many reward designs. Initially, we started with sparse rewards based on the balls being thrown above a certain height. However, we quickly realized, that this would not be enough signal for the agent to learn. After attempting a trial and error based reward design, we found that our results were not improving very much, because our changes would replace one local optima with another rather than converging to our desired result.

To fix this problem, we used a method of analyzing reward functions by considering sample trajectories that might occur and comparing our preferences with the preferences induced by the reward function. For any reward function, a policy that produces an arbitrary trajectory will achieve a return based on the reward function. This return effectively ranks the trajectory for the agent, since it is maximizing the sum of discounted rewards. Therefore, any discrepancies between the human preferences and the preferences induced by the reward function lead to convergence to local optima.

By performing this analysis of ranking potential trajectories by our preferences and then by the returns under our reward function, we were able to remove many of our local optima. This led to a successful result for single ball and simultaneous two ball juggling. We believe that this analysis would be fruitful in any future reward design problems, and highly recommend its adoption.

Our reward function for single ball juggling and simultaneous two ball juggling consisted of the following components:

- Penalties
 - Hand height penalty
 - Out of bounds penalty
 - Fall penalty
 - Excessive joint force penalty
 - Hand XY delta penalty
- Rewards
 - Fingertip reward
 - Lift reward
 - Lift bonus reward
 - Throw reward
 - Catch reward
 - Juggle reward

Hence, the total reward at time t is defined as the weighted sum of all reward and penalty components:

$$R_t = \alpha_1 R_{\text{fingertip},t} + \alpha_2 R_{\text{hand-delta},t} + \alpha_3 R_{\text{lift},t} + \alpha_4 R_{\text{lift-bonus},t} + \alpha_5 R_{\text{juggle},t} + \alpha_6 R_{\text{throw},t} + \alpha_7 R_{\text{catch},t} + \alpha_8 R_{\text{fall},t} + \alpha_9 R_{\text{out-bounds},t} + \alpha_{10} R_{\text{hand-height},t} + \alpha_{11} R_{\text{upward-hand},t} + \alpha_{12} R_{\text{vertical-juggle},t} + \alpha_{13} R_{\text{action-kuka},t} + \alpha_{14} R_{\text{action-allegro},t} \quad (1)$$

The scalars $\alpha_1, \dots, \alpha_{14}$ represent the weight coefficients for each term and are hyperparameters defined in our configuration.

We also utilized environment terminations to enforce desired behavior, combined with a penalty in the reward function. The environment would terminate whenever

- a ball fell below the fall threshold (fall penalty)
- the ball went out of environment bounds (out of bounds penalty)
- the hand went above the height bounds (hand height penalty)

Now, we will examine each component of the dense reward. We use the following notation throughout our reward formulation. The indicator function $\mathbb{1}[P]$ evaluates to 1 if the predicate P is true, and 0 otherwise.

- $f_{j,t}^{xy}$: mean xy fingertip position at time t
- o_t^{xy} : object (ball) xy position
- $h_{i,t}$: object height for ball i
- h_{bonus} : lifting bonus height threshold
- $j_{i,t}$: juggling state indicator for ball i
- $v_{i,t}^z$: vertical velocity of ball i
- $v_{i,t}^*$: target catching velocity for ball i
- p_t^z : palm center z-position
- $\text{indicator}_{i,t}$: binary reward mask for event gating
- C_{max} : clamping maximum (varies by hyperparameter)
- $a_t^{\text{kuka}}, a_t^{\text{allegro}}$: action vectors for the KUKA arm and Allegro hand

$$R_{\text{action-kuka},t} = -\|a_t^{\text{kuka}}\|^2 \quad (2)$$

$$R_{\text{action-allegro},t} = -\|a_t^{\text{allegro}}\|^2 \quad (3)$$

The excessive joint force penalty measured whether the joint torques being commanded were unrealistic, and penalized the policy for generating them.

$$R_{\text{hand-delta},t} = -\|\bar{f}_{t-1}^{xy} - \bar{o}_t^{xy}\| + \|\bar{f}_t^{xy} - \bar{o}_t^{xy}\| \quad (4)$$

The hand delta penalty was a penalty that was meant to keep the hand's xy position near the xy position of all the balls. This was calculated by measuring the change in the distance between the average fingertip xy positions and the ball xy position. Getting closer would yield positive reward, while moving further away would yield negative reward. This encourages the balls to be thrown vertically. It also encourages

the hand to be underneath the ball to be ready for the next catch.

$$R_{\text{fingertip},t} = \sum_i \left(1 - \|\bar{f}_{i,t}^{xy} - o_{i,t}^{xy}\|_2\right) \quad (5)$$

The fingertip reward was based on the closest distance achieved between the fingertips and the ball during the episode. This reward encourages initially grasping the ball which is a prerequisite for any further manipulation.

$$R_{\text{lift},t} = \sum_i \mathbb{1}[h_{i,t} > h_{\text{init}} + \delta] \quad (6)$$

$$R_{\text{lift-bonus},t} = \sum_i \mathbb{1}[h_{i,t} > h_{\text{bonus}}] \cdot \beta \quad (7)$$

The lift reward and lift bonus reward are components that encourage the agent to lift the ball after grasping. The lift reward is a dense reward based on the height off the table. The agent ceases to receive this reward after lifting the ball a certain height off the table. Since this could lead to issues with the agent hovering below the threshold, a large lift bonus reward is given once the threshold is crossed to remove these local optima.

$$R_{\text{throw},t} = \sum_i \mathbb{1} \left[h_{i,t} > h_{\text{bonus}} \wedge v_{i,t}^z > 0 \wedge j_{i,t-1} = 1 \wedge j_{i,t} = 0 \wedge \text{indicator}_{i,t} \right] \quad (8)$$

The throw reward is given based on whether the ball is sufficiently high enough from the palm and whether it has positive upward velocity. The agent receives this reward on the transition from being below the height threshold to being above the height threshold.

$$R_{\text{juggle},t} = \sum_i \mathbb{1} [(j_{i,t-1} = 1) \wedge (j_{i,t} = 0) \wedge \text{indicator}_{i,t}] \quad (9)$$

The juggle reward is given when the ball falls below a specified juggle height. This encourages the agent to throw the ball high enough so that motion appears like a proper juggle.

$$R_{\text{catch},t} = \sum_i \min \left(\max(v_{i,t}^z - v_{i,t}^*, 0), C_{\text{max}} \right) \quad (10)$$

The catching reward is given based on the best upward velocity the ball has achieved within a session. This gets reset every time the ball falls below the juggle height threshold. The idea is to encourage the agent to impart as much energy upwards so that it can execute the next juggle.

$$R_{\text{fall},t} = -\mathbb{1} \left[\min_i h_{i,t} < h_{\text{fall}} \right] \quad (11)$$

$$R_{\text{out-bounds},t} = -\mathbb{1} \left[\exists i : |x_{i,t}| > x_{\text{max}} \vee |y_{i,t}| > y_{\text{max}} \vee h_{i,t} > h_{\text{max}} \right] \quad (12)$$

$$R_{\text{hand-height},t} = -\mathbb{1} \left[\max_j f_{j,t}^z > h_{\text{hand-max}} \right] \quad (13)$$

The fall penalty, out of bounds penalty, and hand height penalty are termination penalties that apply when these termination events occur. The fall penalty is special however, because the threshold used to measure when the event occurs gets moved after the ball falls below the juggle height. It gets moved above the table height, so that the agent cannot utilize the table to achieve degenerate solutions.

The weighted combination of these rewards and penalties produced our successful single and simultaneous double ball juggle results. The weightings were calculated by a rough analysis of the trajectory preferences mentioned above. These weightings can be found in the codebase in the config files. For multiple balls, we simply scaled the rewards/penalties described above. We summed all of the hand delta penalties and fingertip, lift, lift bonus, throw, catch, and juggle rewards for each ball. We triggered the penalties whenever any of the balls violated the boundary conditions or when the hand violated the height boundary.

For alternating two ball juggling, we had to find a way to enforce a schedule for the juggling. We also had to find a way to discourage the two most common convergence modes: simultaneously throwing and catching two balls, or only throwing and catching one of the balls. To do this, we added a termination condition that triggered when the hand was close to both balls.

Hence, we terminate at time t if any of the following hold:

- (1) $\exists i$ s.t. $h_{i,t} < h_{\text{fall}}$ (Fall)
- (2) $\exists i$ s.t. $|x_{i,t}| > x_{\text{max}} \vee |y_{i,t}| > y_{\text{max}} \vee h_{i,t} > h_{\text{env-max}}$ (Out-of-Bounds)
- (3) $\max_j z_{j,t}^{\text{fingertip}} > h_{\text{hand-max}}$ (Hand Height)

We also changed the way we dealt with rewards/penalties in the setting of multiple balls. Instead of aggregating everything, we used a schedule that would only assign rewards if the correct ball was juggled.

The idea behind the schedule was to prevent the agent from juggle a single ball consecutively without juggling another ball. To achieve this, we used a bit mask indicator that tracked whether each ball had been juggled. Whenever a ball was juggled, all the other balls' indicators were turned on while the juggled ball's indicator was turned off. These indicators would then mask the rewards received from throwing, catching, and juggling, forcing the agent to interact with multiple balls.

$$\text{indicator}_{i,t} = \begin{cases} 1, & \text{if } i \neq \text{ball_schedule}_t \\ 0, & \text{if } i = \text{ball_schedule}_t \end{cases} \quad (14)$$

Because of the way that we implemented the switching of these indicators, we were able to encourage a relative timing between the balls. Due to our choice in hyperparameters and implementation, a juggled ball would switch on the other balls' reward bitmask whenever it was at the apex of its juggle. For two balls, this yields an optimal timing, evenly spacing the

balls in the juggle pattern. For three or more balls however, this does not work properly; we observed unstable attempts at juggling.

Moreover, there are many different kinds of schedules that would correspond to unique juggling patterns. Professional jugglers spend a significant amount of time learning these patterns, which is part of the reason this activity is so difficult. We leave it for future work to determine a better way of specifying schedules to encourage different juggling patterns.

$$R_{\text{hand-delta},t} = -\left\| \bar{f}_t^{xy} - o_{\text{focus},t}^{xy} \right\|_2 + \left\| \bar{f}_{t-1}^{xy} - o_{\text{focus},t}^{xy} \right\|_2 \quad (15)$$

$$\text{focus} = \begin{cases} \arg \min_i h_{i,t}, & \text{if any } i \text{ not yet lifted} \\ \arg \min_i \tau_{i,t}, & \text{otherwise} \end{cases} \quad (16)$$

$$\tau_{i,t} = \frac{v_{i,t}^z + \sqrt{(v_{i,t}^z)^2 + 2gh_{i,t}}}{g} \quad (17)$$

Finally, we modified the hand delta penalty term to only focus on the ball that would fall the earliest based on its velocity and current height. This was calculated using a basic kinematic formula, and proved very effective at positioning the hand for its next catch. This modification however allowed for less vertical throws that result in termination due to balls going out of bounds. As a result, we added a termination condition based on whether the XY position of any ball was outside the range of the table.

With these modifications, we were able to achieve alternating two ball juggling.

V. EXPERIMENTS

A. Results

We were able to achieve strong juggling results for single ball juggling, simultaneous two ball juggling, and alternating two ball juggling. These juggles were stable, continuous, and almost human-like, which is what we sought to achieve. We were also able to achieve an unstable 3 ball juggle using the reward function we built for the 2 ball alternating juggle. These results can be found in the JugglingVideos/Working. More demonstrations of other variants we discovered through the process can be found in JugglingVideos/Bloopers.

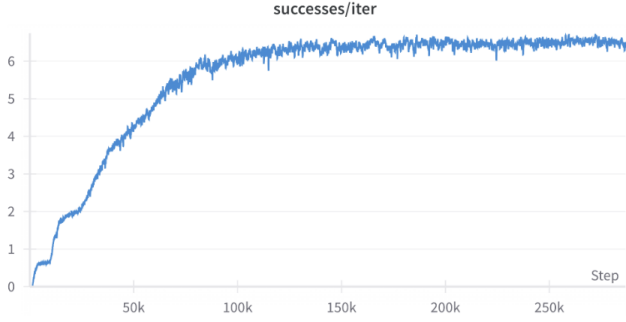


Fig. 3: Number of successful juggling attempts per iteration over the course of training, where a successful attempt is defined by an instance of a ball entering the established juggling zone

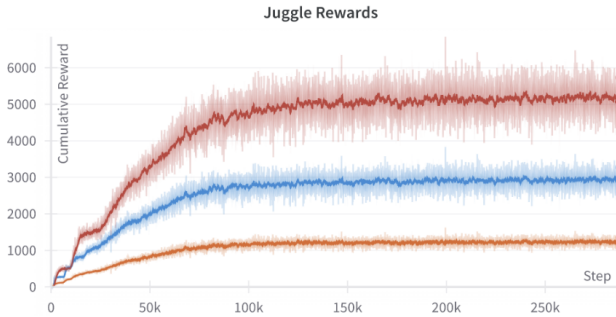


Fig. 4: Cumulative reward across training steps for different reward components. Throw reward (blue), catch reward (orange), and juggle reward (red) plateau after 150k steps.

B. Emergent behaviors

One of the most compelling emergent behaviors observed was the agent’s development of upward hand motion during juggling. Rather than merely reacting to falling balls, the policy learned to lift the hand vertically after each catch. This anticipatory action aligns the palm underneath the next target, increasing the success rate of subsequent throws. This behavior was not explicitly encouraged by the reward function but emerged as a strategy to maximize long-term return, especially under the vertical-juggle and catch-velocity reward terms. We find this interesting since we can draw parallels from this behavior to human learned methods of juggling, which are also palm up.

C. Limitations

Despite strong performance on one and two-ball tasks, the policy failed to generalize to three-ball juggling. We hypothesize that this is due to increased scheduling complexity. In the two-ball sequential setting, the agent benefits from alternating interaction, with one ball in the air while the other is being caught. However, in the three-ball case, the juggling rhythm requires finer precision, which we hypothesize our bitmask-based reward indicators and current policy architecture could

not handle effectively. This suggests a need for either curriculum learning or architectural modifications to scale to higher ball counts.

For alternating two ball juggling, we had to reduce the gravity in order for it to learn. We leave it to future work to make the alternating juggling work for more balls. Additionally, the reward shaping is very aggressive and took a long time to engineer properly. Throughout the process, we found many ways to not juggle. It would be interesting to explore better ways of specifying the task to make this process easier.

VI. CONCLUSION

In this project, we explored the feasibility of training a robotic system to juggle using reinforcement learning in a high-fidelity simulation environment. Through extensive experimentation mainly in the realm of reward shaping, we achieved successful one-ball and two-ball juggling behaviors, with sequential two-ball juggling (throwing in order) producing the most stable and sustained results. Our approach required carefully engineered reward functions and environment terminations to encourage rhythmic, repeatable behaviors while penalizing instability and unrealistic control signals.

Although we attempted three-ball juggling, our policy failed to achieve consistent stabilization, highlighting both the exponential complexity of multi-object dexterous control and the limits of PPO in this setting. Nevertheless, our results show that even standard RL algorithms—when paired with high-throughput simulation and thoughtful environment design—can enable complex motor skills such as juggling from scratch.

Future work may involve scaling to three or more objects using hierarchical control or population-based training methods like DexPBT, incorporating richer sensory input such as vision, or transferring learned juggling policies to real-world hardware through sim-to-real techniques and domain randomization. This project offers both a technical benchmark and a platform for further exploration into general-purpose dexterous skill acquisition in robotics.

CONTRIBUTIONS

The project work as described in our paper was divided between Rhea Rai and Abhinav Peri, with both authors contributing to the design, implementation, experimentation, and writing of the report.

Dr. Brad Knox (Research Associate Professor, UT Austin) provided advisory input on reward shaping and analysis. His expertise helped refine our reward function to better guide the learning process and interpret trajectory outcomes.

This project builds on the Isaac Gym simulator and codebase provided by NVIDIA. We build the simulator from source with our own custom modifications to support the juggling environments. [4].

Our full codebase and training framework are publicly available at <https://github.com/AbhinavPeri/JugglingIsaacGymEnvs>.

REFERENCES

- [1] Jason Chemin and Jehee Lee. A physics-based juggling simulation using reinforcement learning. *Proceedings of the 11th ACM SIGGRAPH Conference on Motion, Interaction and Games*, pages 1–7, nov 2018. doi: 10.1145/3274247.3274516.
- [2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, aug 2018. doi: 10.48550/arXiv.1801.01290.
- [3] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, jul 2019. doi: 10.48550/arXiv.1509.02971.
- [4] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, aug 2021. doi: 10.48550/arXiv.2108.10470.
- [5] OpenAI, Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafał Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *CoRR*, 2018. URL <http://arxiv.org/abs/1808.00177>.
- [6] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik’s cube with a robot hand. *arXiv preprint*, 2019.
- [7] Aleksei Petrenko, Arthur Allshire, Gavriel State, Ankur Handa, and Viktor Makoviychuk. Dexpbt: Scaling up dexterous manipulation for hand-arm systems with population based training. *arXiv preprint arXiv:2305.12127*, may 2023. doi: 10.48550/arXiv.2305.12127.
- [8] Kai Ploeger, Michael Lutter, and Jan Peters. High acceleration reinforcement learning for real-world juggling with binary rewards. *arXiv preprint arXiv:2010.13483*, oct 2020. doi: 10.48550/arXiv.2010.13483.
- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, aug 2017. doi: 10.48550/arXiv.1707.06347.
- [10] Kazutoshi Tanaka, Masashi Hamaya, Devwrat Joshi, Felix von Drigalski, Ryo Yonetani, Takamitsu Matsubara, and Yoshihisa Ijiri. Learning robotic contact juggling. *Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 958–964, sep 2021. doi: 10.1109/IROS51168.2021.9636790.
- [11] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, oct 2012. ISSN 2153-0866. doi: 10.1109/IROS.2012.6386109.
- [12] Yuke Zhu, Josiah Wong, Ajay Mandlekar, and Roberto Martín-Martín. robosuite: A modular simulation framework and benchmark for robot learning. *CoRR*, abs/2009.12293, 2020. URL <https://arxiv.org/abs/2009.12293>.